Лабораторная работа № 3

Титульный лист

Дисциплина: Объектно-ориентированное программирование
Выполнил: Мостовщиков Владимир Витальевич
Группа: 6204-010302D
Преподаватель: Борисов Дмитрий Сергеевич
Год: 2025

Содержание:

1. Задание 1. Изучение стандартных исключений Java
2. Задание 2. Создание собственных исключений
3. Задание 3. Доработка ArrayTabulatedFunction (ранее TabulatedFunction) с выбрасыванием исключений
4. Задание 4. Реализация LinkedListTabulatedFunction и внутренних операций списка
5. Задание 5. Реализация методов табулированной функции на связном списке
6. Задание 6. Интерфейс TabulatedFunction и реализация в двух классах
7. Задание 7. Проверка работы в Main: сценарии и исключения

Задание 1. Изучение стандартных исключений Java

Для выполнения первого задания я изучил классы из API Java: Exception, IndexOutOfBoundsException, ArrayIndexOutOfBoundsException, IllegalArgumentException, IllegalStateException. Назначение и сценарии использования понятны и применены в лабораторной работе.

Задание 2. Создание собственных исключений

В пакете functions я реализовал два класса:

• FunctionPointIndexOutOfBoundsException — наследник IndexOutOfBoundsException; используется при ошибках индекса при обращении к точкам.

• InappropriateFunctionPointException — наследник Exception; используется при нарушении упорядоченности точек или попытке добавить точку с повторяющейся x.

```java
package functions;

/* Появляется при попытке обратиться к несуществующему индексу */
public class FunctionPointIndexOutOfBoundsException extends IndexOutOfBoundsException {
    public FunctionPointIndexOutOfBoundsException() {  6 usages  new *
        super();
    }

    public FunctionPointIndexOutOfBoundsException(String s) {  9 usages  new *
        super(s);
    }
}
```

Рисунок 1 – Реализация класса FunctionPointIndexOutOfBoundsException

```java
package functions;

// Появляется при попытке добавить или изменить точку функции некорректным образом
/* (например, x выходит из интервала между соседями или дублируется x). */
public class InappropriateFunctionPointException extends Exception {  29 usages  new *
    public InappropriateFunctionPointException() {  8 usages  new *
        super();
    }

    public InappropriateFunctionPointException(String message) {  20 usages  new *
        super(message);
    }
}
```

Рисунок 2 – реализация класса InappropriateFunctionPointException

Задание 3. Доработка ArrayTabulatedFunction

Для выполнения задачи в класс ArrayTabulatedFunction внесены проверки и выбрасывание исключений согласно заданию, так же было доработанно сравнение вещественных чисел, были созданы вспомогательные статические методы для сравнения вещественных чисел.

- Конструкторы: IllegalArgumentException при leftX >= rightX или числе точек < 2.
- Методы getPoint(), setPoint(), getPointX(), setPointX(), getPointY(), setPointY(), deletePoint(): бросают FunctionPointIndexOutOfBoundsException при неверном индексе.
- Методы setPoint() и setPointX(): бросают InappropriateFunctionPointException при нарушении порядка по x.
- Метод addPoint(): бросает InappropriateFunctionPointException при дублировании x.

Метод deletePoint(): бросает IllegalStateException при попытке удалить точку, когда их стало < 3.

```
private static final double EPSILON = 1e-9;  5 usages

private static boolean eq(double a, double b) { return Math.abs(a - b) <= EPSILON; }
private static boolean lt(double a, double b) { return a <  b - EPSILON; }  5 usages  n
private static boolean gt(double a, double b) { return a >  b + EPSILON; }  4 usages  n
private static boolean le(double a, double b) { return a <= b + EPSILON; }  1 usage  ne
private static boolean ge(double a, double b) { return a >= b - EPSILON; }  1 usage  ne
```

```java
// Конструктор [leftX; rightX], pointsCount точек, все y=0
public ArrayTabulatedFunction(double leftX, double rightX, int pointsCount) {
    if (rightX <= leftX) {
        throw new IllegalArgumentException("leftX must be < rightX");
    }
    if (pointsCount < 2) {
        throw new IllegalArgumentException("pointsCount must be >= 2");
    }
    this.size = pointsCount;
    this.points = new FunctionPoint[Math.max(pointsCount, 2)];

    // // Равномерный шаг по x
    double step = (rightX - leftX) / (pointsCount - 1);
    for (int i = 0; i < pointsCount; ++i) {
        double x = leftX + i * step;
        points[i] = new FunctionPoint(x, y: 0.0);
    }
}
```

```java
// Конструктор: [leftX; rightX], значения y — из массива
public ArrayTabulatedFunction(double leftX, double rightX, double[] values) {
    if (rightX <= leftX) {
        throw new IllegalArgumentException("leftX must be < rightX");
    }
    if (values == null || values.length < 2) {
        throw new IllegalArgumentException("values length must be >= 2");
    }
    this.size = values.length;
    this.points = new FunctionPoint[Math.max(size, 2)];

    double step = (rightX - leftX) / (size - 1);
    for (int i = 0; i < size; ++i) {
        double x = leftX + i * step;
        points[i] = new FunctionPoint(x, values[i]);
    }
}
```

Рисунок 3,4 – изменения в реализации конструкторов

```java
@Override  4 usages  Metilka
public FunctionPoint getPoint(int index) {
    requireIndex(index);
    return new FunctionPoint(points[index]);
}
// Заменяет точку по индексу на переданную, Исправлено теперь сравнение с машинным эпсилон
@Override  5 usages  Metilka *
public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {
    if (point == null) return;
    requireIndex(index);
    double newX = point.getX();
    if (index > 0) {
        double leftX = points[index - 1].getX();
       if (!gt(newX, leftX)) throw new InappropriateFunctionPointException("x must be > left neighbor");
    }
    if (index < size - 1) {
        double rightX = points[index + 1].getX();
         if (!lt(newX, rightX)) throw new InappropriateFunctionPointException("x must be < right neighbor
    }
    points[index] = new FunctionPoint(point); // копия (инкапсуляция)
}
```

Рисунок 5 - Реализация методов getPoint(), setPoint()(исправлено)

```
@Override 9 usages  Metilka *
public void setPointX(int index, double x) throws InappropriateFunctionPointException {
    requireIndex(index);
    if (size == 1) {
        points[0].setX(x);
        return;}
    if (index == 0) {
        double nextX = points[1].getX();
        if (!lt(x, nextX)) throw new InappropriateFunctionPointException("x must be < right neighbor");
        points[0].setX(x);
        return;}
    if (index == size - 1) {
        double prevX = points[size - 2].getX();
        if (!gt(x, prevX)) throw new InappropriateFunctionPointException("x must be > left neighbor");
        points[size - 1].setX(x);
        return;}
    double leftX = points[index - 1].getX();
    double rightX = points[index + 1].getX();
    if (!(gt(x, leftX) && lt(x, rightX))) {
        throw new InappropriateFunctionPointException("x must be strictly between neighbors");}
    points[index].setX(x);
}
```

Рисунок 6 – Реализация исправленного метода setPointX

```
//  Абсцисса x точки по индексу
@Override  34 usages  Metilka
public double getPointX(int index) {
    requireIndex(index);
    return points[index].getX();
}
```

Рисунок 7 – Реализация метода getPointX

```java
@Override  5 usages  new *
public double getPointY(int index) {
    requireIndex(index);
    return points[index].getY();
}
//Меняет у точки по индексу. Порядок по x не затрагивается
@Override  3 usages  new *
public void setPointY(int index, double y) {
    requireIndex(index);
    points[index].setY(y);
}
   //   Удаляет точку по индексу со сдвигом хвоста влево.
  //    Минимально допустимое число точек равно 3.
@Override  3 usages  new *
public void deletePoint(int index) {
    requireIndex(index);
    if (size < 3) throw new IllegalStateException("cannot delete when points count < 3");
    System.arraycopy(points, srcPos: index + 1, points, index, length: size - index - 1);
    points[--size] = null;
}
```

Рисунок 8 – реализация методов getPointY(), setPointY(), deletePoint()

```java
//  Теперь сравнение с машинным эпсилон
@Override  7 usages  Metilka *
public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {
    if (point == null) return;
    double x = point.getX();
    int insertIndex = 0;
    while (insertIndex < size && lt(points[insertIndex].getX(), x)) {insertIndex++;}
    if (insertIndex < size && eq(points[insertIndex].getX(), x)) {
        throw new InappropriateFunctionPointException("duplicate x");
    }

    if (size >= points.length) {
        int newCapacity = (points.length == 0) ? 2 : (points.length * 3 / 2 + 1);
        FunctionPoint[] newArray = new FunctionPoint[newCapacity];
        if (size > 0) System.arraycopy(points, srcPos: 0, newArray, destPos: 0, size);
        points = newArray;}
        // Сдвигаем хвост вправо, чтобы освободить позицию insertIndex
    if (insertIndex < size) {System.arraycopy(points, insertIndex, points, destPos: insertIndex + 1, length: size - insertIndex);}
    // Вставляем копию точки
    points[insertIndex] = new FunctionPoint(point); // копия
    size++;
}
```

Рисунок 9 – реализация исправленного метода addPoint()

Теперь программа будет информировать пользователя об ошибках, выбрасывая исключения.

Задание 4. Реализация LinkedListTabulatedFunction и внутренних операций

В ходе выполнения 4 задания я реализовал двусвязный циклический список с выделенной головой. Внутренний класс FunctionNode хранит данные точки и ссылки prev/next; внешним пользователям ссылки не выдаются. Так же были проделаны изменения аналогичные заданию 3.

- Базовые операции списка:
- getNodeByIndex(int index) — доступ к i-му элементу с выбором направления обхода (от головы или от хвоста).
- addNodeToTail() — добавление элемента в конец последовательности значимых точек.
- addNodeByIndex(int index) — вставка элемента по индексу (0..size).
- deleteNodeByIndex(int index) — удаление элемента по индексу.

```
private static class FunctionNode {  23 usages  new *
    FunctionPoint point;  28 usages
    FunctionNode prev;  18 usages
    FunctionNode next;  24 usages

    FunctionNode() {  1 usage  new *
    }

    FunctionNode(FunctionPoint p) {  2 usages  new *
        this.point = p;
    }
}

// Данные в head не хранятся. Он упрощает вставки и удаления
private final FunctionNode head;  17 usages


private int size; // Тенкущее количество значимых точек  21 usages
```

Рисунок 10 – реализация FunctionNode

```
private FunctionNode getNodeByIndex(int index) {  8 usages  new *
    if (index < 0 || index >= size) {
        throw new FunctionPointIndexOutOfBoundsException("index=" + index);
    }
    if (index < size / 2) {
        FunctionNode cur = head.next;
        for (int i = 0; i < index; i++) cur = cur.next;
        return cur;
    } else {
        FunctionNode cur = head.prev;
        for (int i = size - 1; i > index; i--) cur = cur.prev;
        return cur;
    }
}
```

Рисунок 11 - Реализация метода getNodeByIndex()

```
private FunctionNode addNodeToTail(FunctionPoint p) {  3 usages  new *
    FunctionNode last = head.prev;
    FunctionNode node = new FunctionNode(new FunctionPoint(p)); // копия (инкапсуляция)

    node.prev = last;
    node.next = head;
    last.next = node;
    head.prev = node;

    size++;
    return node;
}
```

Рисунок 12 - Реализация метода addNodeToTail()

```
private FunctionNode addNodeByIndex(int index, FunctionPoint p) {  1 usage  ne
    if (index < 0 || index > size) {
        throw new FunctionPointIndexOutOfBoundsException("index=" + index);
    }
    if (index == size) {
        return addNodeToTail(p);
    }
    FunctionNode at = getNodeByIndex(index);
    FunctionNode before = at.prev;
    FunctionNode node = new FunctionNode(new FunctionPoint(p)); // копия

    node.prev = before;
    node.next = at;
    before.next = node;
    at.prev = node;

    size++;
    return node;
}
```

Рисунок 13 - Реализация метода addNodeByIndex()

```
private FunctionNode deleteNodeByIndex(int index) {
    FunctionNode at = getNodeByIndex(index);
    at.prev.next = at.next;
    at.next.prev = at.prev;
    size--;
    // обнулим ссылки удалённого узла
    at.next = null;
    at.prev = null;
    return at;
}
```

Рисунок 14 – реализация метода deleteNodeByIndex()

Задание 5. Методы табулированной функции на связном списке

Я создал три конструктора для класса LinkedListTabulatedFunction:

- Базовый конструктор - создает пустой список с фиктивной головой
- Конструктор с равномерной разметкой - создает функцию с заданным количеством равномерно распределенных точек
- Конструктор с массивом значений - создает функцию с значениями y из переданного массива

Конструкторы и методы представляют собой идентичный набор методов с одинаковыми сигнатурами, что и ArrayTabulatedFunction и выбрасывают те же исключения. Линейная интерполяция: getFunctionValue(x) возвращает NaN вне границ, при совпадении соответствующий y, иначе выполняется интерполяция по ближайшему отрезку.

- Ключевые проверки:
- setPoint()/setPointX() — новая абсцисса должна строго лежать между соседями.
- addPoint() — запрет добавления точки с существующей абсциссой.
- deletePoint() — запрет удаления, если после операции останется менее трёх точек.

```
//Пустой список ^ head указывает сам на себя
public LinkedListTabulatedFunction() {  24 usages
    head = new FunctionNode();
    head.next = head;
    head.prev = head;
    size = 0;
}
```

Рисунок 15 – Базовый конструктор

```java
public LinkedListTabulatedFunction(double leftX, double rightX, int pointsCount) {
    this();
    if (rightX <= leftX) {
        throw new IllegalArgumentException("leftX must be < rightX");
    }
    if (pointsCount < 2) {
        throw new IllegalArgumentException("pointsCount must be >= 2");
    }
    double step = (rightX - leftX) / (pointsCount - 1);
    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        addNodeToTail(new FunctionPoint(x, y: 0.0));
    }
}
```

Рисунок 16 - Конструктор с равномерной разметкой

```java
public LinkedListTabulatedFunction(double leftX, double rightX, double[] values) {
    this();
    if (rightX <= leftX) {
        throw new IllegalArgumentException("leftX must be < rightX");
    }
    if (values == null || values.length < 2) {
        throw new IllegalArgumentException("values length must be >= 2");
    }
    double step = (rightX - leftX) / (values.length - 1);
    for (int i = 0; i < values.length; i++) {
        double x = leftX + i * step;
        addNodeToTail(new FunctionPoint(x, values[i]));
    }
}
```

Рисунок 17 – Конструктор с значениями y из переданного массива

```
@Override  5 usages  Metilka *
public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {
    if (point == null) return;
    FunctionNode node = getNodeByIndex(index);
    double newX = point.getX();

    if (index > 0) {
        double leftX = node.prev.point.getX();
        if (!gt(newX, leftX)) {
            throw new InappropriateFunctionPointException("x must be > left neighbor");
        }
    }
    if (index < size - 1) {
        double rightX = node.next.point.getX();
        if (!lt(newX, rightX)) {
            throw new InappropriateFunctionPointException("x must be < right neighbor");
        }
    }
    node.point = new FunctionPoint(point); // копия
}
```

Рисунок 18 – Реализация setPoint() (с правками)

```
@Override  9 usages  Metilka *
public void setPointX(int index, double x) throws InappropriateFunctionPointException {
    FunctionNode node = getNodeByIndex(index);
    if (size == 1) {
        node.point.setX(x);
        return;}
    if (index == 0) {
        double nextX = node.next.point.getX();
        if (!lt(x, nextX)) throw new InappropriateFunctionPointException("x must be < right neighbor");
        node.point.setX(x);
        return;}
    if (index == size - 1) {
        double prevX = node.prev.point.getX();
        if (!gt(x, prevX)) throw new InappropriateFunctionPointException("x must be > left neighbor");
        node.point.setX(x);
        return;}
    double leftX = node.prev.point.getX();
    double rightX = node.next.point.getX();
    if (!(gt(x, leftX) && lt(x, rightX))) {
        throw new InappropriateFunctionPointException("x must be strictly between neighbors");}
    node.point.setX(x);}
```

Рисунок 19 – Реализация setPointX() с изменениями

```
@Override  3 usages  new *
public void deletePoint(int index) {
    if (size < 3) {
        throw new IllegalStateException("cannot delete when points count < 3");
    }
    deleteNodeByIndex(index);
}
```

Рисунок 20 - Реализация deletePoint()

```
// Сравниваем с машинным эпсилон
@Override  7 usages  Metilka *
public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {
    if (point == null) return;

    double x = point.getX();

    int insertIndex = 0;
    FunctionNode cur = head.next;
    while (insertIndex < size && lt(cur.point.getX(), x)) {
        cur = cur.next;
        insertIndex++;
    }
    // дубликат x запрещён
    if (insertIndex < size && eq(cur.point.getX(), x)) {
        throw new InappropriateFunctionPointException("duplicate x");
    }
    // вставка копии точки в найденную позицию
    addNodeByIndex(insertIndex, point);
}
```

Рисунок 21 – реализация addPoint() с исправлениями

Задание 6. Интерфейс TabulatedFunction

Для выполнения задания я реализовал интерфейс TabulatedFunction, задающий общую спецификацию методов. Классы ArrayTabulatedFunction и

LinkedListTabulatedFunction реализуют данный интерфейс, что позволяет подменять реализацию без изменения кода, работающего с функцией.

```
package functions;

public interface TabulatedFunction {  4 usages  2 implementations  new *
    int getPointsCount(); // Возвращает количество точек в табулированной функции  7 usages  2 implementations  new *
    double getLeftDomainBorder(); //Возвращает левую границу области определения функции  1 usage  2 implementations  new *
    double getRightDomainBorder();// Возвращает правую границу области определения функции  1 usage  2 implementations  new *

    double getFunctionValue(double x); // Вычисляет значение функции в заданной точке x  2 usages  2 implementations  new *

    FunctionPoint getPoint(int index); // Возвращает точку функции по указанному индексу  3 usages  2 implementations  new *

    void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException; //Замена точки по индексу

    double getPointX(int index); //Возвращает x-координату точки по указанному индексу  13 usages  2 implementations  new *
    // Устанавливает x-координату точки по указанному индексу
    void setPointX(int index, double x) throws InappropriateFunctionPointException;  3 usages  2 implementations  new *

    double getPointY(int index);  5 usages  2 implementations  new *
    void setPointY(int index, double y);  3 usages  2 implementations  new *

    void deletePoint(int index);  3 usages  2 implementations  new *

    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;  3 usages  2 implementations  new *
}
```

Рисунок 22 – реализация интерфейса TabulatedFunction

Задание 7. Проверка работы в Main

Для выполнения задания 7 создаем в пакете по умолчанию класс Main. В классе Main (файл Main.java) реализованы два сценария: (1) базовая демонстрация работы функций и интерполяции, (2) тестирование всех заданных исключений. Переменная объявлена типа TabulatedFunction; для проверки второй реализации достаточно подменить класс при создании объекта.

Общие проверки для обеих реализаций:

- Табулирование и заполнение значениями: пример линейной функции y = 5x + 10 (для массива) и квадратичной y = 3x² - 2 (для списка).
- Интерполяция:
  - значения внутри диапазона,
  - значения ровно на границах (x == left/right),
  - значения вне диапазона (ожидается NaN).
- Операции с точками:
  - чтение/изменение y (getPointY/setPointY),
  - безопасная замена точки setPoint и изменение абсциссы setPointX при сохранении строгого порядка x,

    - инкапсуляция: getPoint(i) возвращает копию
    - удаление точки, в т.ч. с индекса 0 и последнего индекса (для списка).

Проверки с учётом машинного эпсилон

- addPoint:
    - явный дубликат по абсциссе x приводит к InappropriateFunctionPointException;
    - почти-дубликат: новое значение x отличается от существующего меньше, чем выбранная машинная погрешность в реализации, тоже приводит к InappropriateFunctionPointException;
    - если разница по x больше выбранной машинной погрешности, вставка проходит успешно.
- setPointX и setPoint на границе допустимого:
    - для первой точки: значение x, практически совпадающее с x соседа справа, трактуется как нарушение требования строго меньше;
    - для последней точки: значение x, практически совпадающее с x соседа слева, трактуется как нарушение требования строго больше.

Специальные проверки LinkedListTabulatedFunction

- Вставка в начало: addPoint с x меньше текущей левой границы корректно вставляет новую нулевую точку, индексы сдвигаются. Затем выполняется удаление нулевой точки и проверка новой левой границы и индексации.
- Вставка в конец: addPoint с x больше текущей правой границы корректно добавляет точку в хвост. Затем выполняется удаление последней точки и проверяется связность и индексация.
- Интерполяция на границах: вычисление значений ровно на левой и правой границе после модификаций у головы и хвоста.

Проверяемые исключения (обе реализации)

- IllegalArgumentException в конструкторах:
    - левая граница больше или равна правой;
    - количество точек меньше двух;
    - при создании по массиву значений длина массива меньше двух.
- FunctionPointIndexOutOfBoundsException:
    - выход за границы индекса в getPoint, getPointX, getPointY, setPointY.
- InappropriateFunctionPointException:
    - нарушение строгого порядка абсцисс при setPointX и setPoint;

    - дубликат и почти-дубликат абсциссы при addPoint.
- IllegalStateException:
    - попытка удалить точку, если после удаления останется менее двух точек (то есть текущее количество точек меньше трех).

```
IAE(Array): левая >= правая  leftX must be < rightX
IAE(Array): pointsCount < 2  pointsCount must be >= 2
IAE(List): левая >= правая  leftX must be < rightX
IAE(List): pointsCount < 2  pointsCount must be >= 2
IAE(Array): values.length < 2  values length must be >= 2
IAE(List): values.length < 2  values length must be >= 2
```

Рисунок 23 - Проверка конструкторов: выбрасывание IllegalArgumentException

```
 ArrayTabulatedFunction: исходные точки (y = 5x+10)
точка 0: x=0,00, y=10,00
точка 1: x=2,00, y=20,00
точка 2: x=4,00, y=30,00
точка 3: x=6,00, y=40,00
точка 4: x=8,00, y=50,00
точка 5: x=10,00, y=60,00
границы: [0,00; 10,00]
```

Рисунок 24 - Исходная таблица значений для ArrayTabulatedFunction (y = 5x + 10)

```
Значения функции:
f(-1,0) = NaN
f( 0,0) = 10,00
f( 1,0) = 15,00
f( 3,0) = 25,00
f( 5,5) = 37,50
f(10,0) = 60,00
f(11,0) = NaN
```

Рисунок 25 - Проверка getFunctionValue(x) (интерполяция и значения вне диапазона)

```
Тест геттеров/сеттеров:
до setPointY: y[2]=30,00
после setPointY: y[2]=999,00
после setPointX: x[2]=4,00 (между 2,00 и 6,00)
 Inappropriate(setPointX): x must be strictly between neighbors
после setPoint: #1 -> x=1,00, y=15,00
 Inappropriate(setPoint): x must be < right neighbor
после addPoint(7.5, f(7.5)) количество = 7
 Inappropriate(addPoint дубликат): duplicate x
после deletePoint(0) количество = 6
 IllegalState(delete при <3 точках): cannot delete when points count < 3
 FPIndexOOBE(getPoint -1): index=-1
 FPIndexOOBE(getPoint 99): index=99
```

Рисунок 26 - Тесты модификации и защитных проверок (Демонстрирует:

- Работу геттеров и сеттеров
- Обработку InappropriateFunctionPointException при нарушении порядка точек
- Обработку IllegalStateException при попытке удалить точку при количестве < 3
- Обработку FunctionPointIndexOutOfBoundsException при неверных индексах)

```
 LinkedListTabulatedFunction: точки
точка 0: x=-2,00, y=10,00
точка 1: x=0,00, y=-2,00
точка 2: x=2,00, y=10,00
точка 3: x=4,00, y=46,00
 Значения в выборочных x
g(-3,0) = NaN
g(-1,0) = 4,00
g( 0,0) = -2,00
g( 1,5) = 7,00
g( 5,0) = NaN
 Inappropriate(list/addPoint дубликат): duplicate x
 Inappropriate(list/setPointX): x must be strictly between neighbors
 FPIndexOOBE(list/getPoint 999): index=999
```

Рисунок 27 - Проверка LinkedListTabulatedFunction печать точек(для функции y = 3x^2-2), интерполяция и исключения

```
После addPoint(newX<left):
  0: x=-3.0, y=25.0
  1: x=-2.0, y=10.0
  2: x=0.0, y=-2.0
  3: x=2.0, y=10.0
  4: x=4.0, y=46.0
новая левая граница: -3.0

После addPoint(x>right):
  0: x=-3.0, y=25.0
  1: x=-2.0, y=10.0
  2: x=0.0, y=-2.0
  3: x=2.0, y=10.0
  4: x=4.0, y=46.0
  5: x=5.0, y=73.0
новая правая граница: 5.0
```

Рисунок 28 - Вставка точки в начало и в конец списка

```
После deletePoint(0):
  0: x=-2.0, y=10.0
  1: x=0.0, y=-2.0
  2: x=2.0, y=10.0
  3: x=4.0, y=46.0
  4: x=5.0, y=73.0
левая граница: -2.0
ожидалось: количество=5, новая нулевая=(-2.0, 10.0)

После deletePoint(last):
  0: x=-2.0, y=10.0
  1: x=0.0, y=-2.0
  2: x=2.0, y=10.0
  3: x=4.0, y=46.0
ожидалось: новый последний=(4.0, 46.0)
```

Рисунок 29 - Удаление нулевой и последней точки

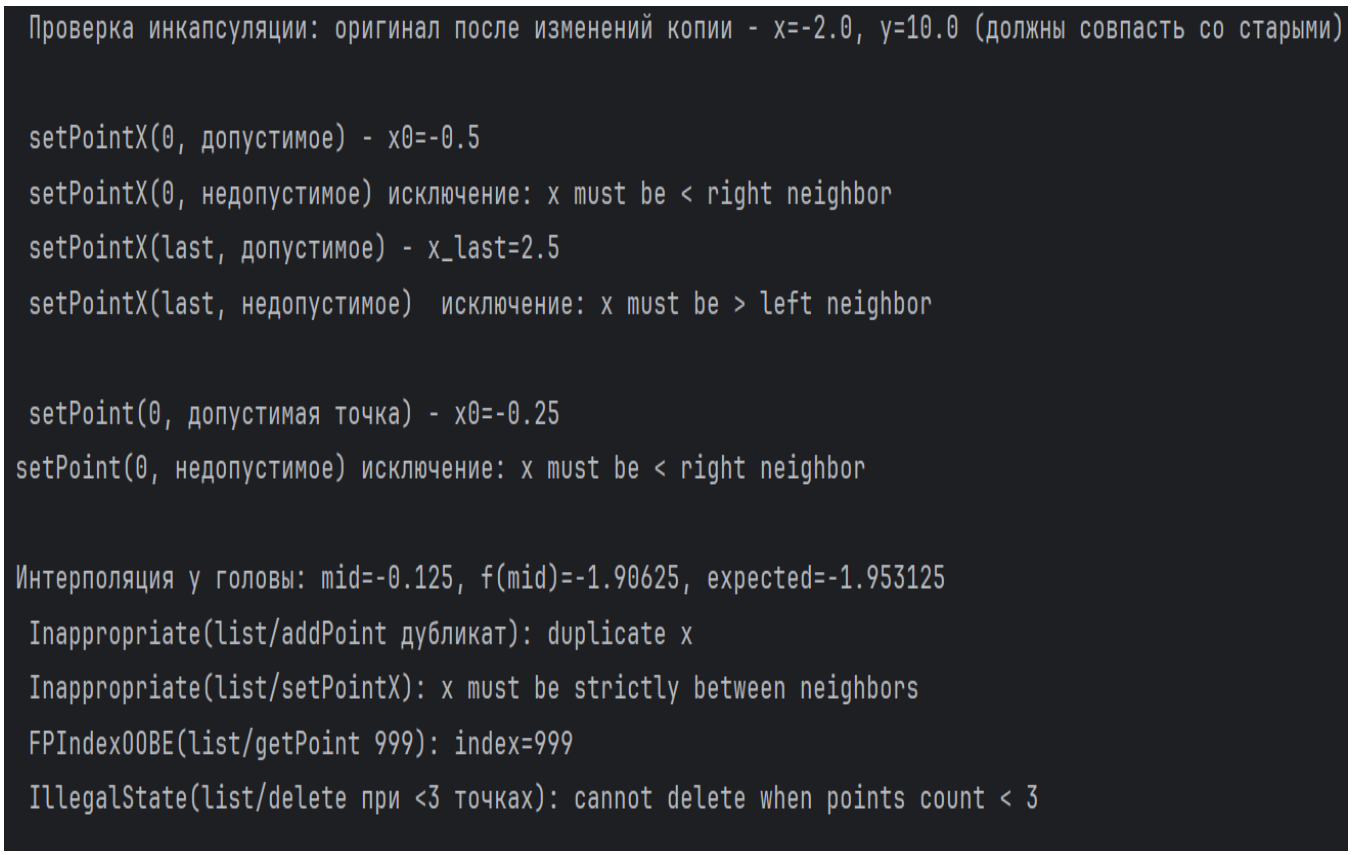


Рисунок 30 - Дополнительные проверки корректности и исключений